

数据结构

王翔羚

清华大学

2026 年 5 月 3 日



- ① 闲话
- ② Prob1-4
- ③ Prob5-8
- ④ Prob9-11

- ① 闲话
- ② Prob1-4
- ③ Prob5-8
- ④ Prob9-11

- 数据结构是各种大赛当中非常常见的考点，并且绝大多数的题目都不以数据结构来区分。

- 数据结构是各种大赛当中非常常见的考点，并且绝大多数的题目都不以数据结构来区分。
- 所以熟练掌握数据结构还是挺重要的。

- 数据结构是各种大赛当中非常常见的考点，并且绝大多数的题目都不以数据结构来区分。
- 所以熟练掌握数据结构还是挺重要的。
- 选的题都不是很难，希望大家都能参与进来。

- ① 闲话
- ② Prob1-4
- ③ Prob5-8
- ④ Prob9-11

QOJ9492. 树上简单求和

- 有 n 个节点，第 i 个节点的点权是 a_i 。
- 有两棵树分别独立地连接了这 n 个点，两棵树点权是共用的。
- 一共需要进行 m 次操作，每次操作给定 x, y, k ，操作分为以下两步：
 1. 将第一棵树上 x, y 之间最短路径上所有节点的点权增加 k 。
 2. 求第二棵树上 x, y 之间最短路径上的点权和对 2^{64} 取模之后的结果。
- 数据范围： $n, m \leq 2 \times 10^5, k, a_i \leq 2^{64}$ 。

QOJ9492. 树上简单求和

- 首先考虑在第二棵树上随机 $\sqrt{n}$ 个关键点，这样可以保证所有点到最近的关键点祖先步数 $\sqrt{n}$ 。

QOJ9492. 树上简单求和

- 首先考虑在第二棵树上随机 $\sqrt{n}$ 个关键点，这样可以保证所有点到最近的关键点祖先步数 $\sqrt{n}$ 。
- 然后我们考虑所有查询，将 x, y 跳到最近的关键点，而路径上经过的点考虑直接进行单点查询，于是每次询问会询问 $\Theta(\sqrt{n})$ 个单点。

QOJ9492. 树上简单求和

- 首先考虑在第二棵树上随机 $\sqrt{n}$ 个关键点，这样可以保证所有点到最近的关键点祖先步数 $\sqrt{n}$ 。
- 然后我们考虑所有查询，将 x, y 跳到最近的关键点，而路径上经过的点考虑直接进行单点查询，于是每次询问会询问 $\Theta(\sqrt{n})$ 个单点。
- 询问单点的方法，首先把修改差分，于是第二棵树上的单点询问就变成了询问第一棵树的子树和，在第一棵树上把 dfs 序分成 $\sqrt{n}$ 个区间，然后维护一个 $\Theta(\sqrt{n}) - \Theta(1)$ 的数据结构即可。

QOJ9492. 树上简单求和

- 首先考虑在第二棵树上随机 $\sqrt{n}$ 个关键点，这样可以保证所有点到最近的关键点祖先步数 $\sqrt{n}$ 。
- 然后我们考虑所有查询，将 x, y 跳到最近的关键点，而路径上经过的点考虑直接进行单点查询，于是每次询问会询问 $\Theta(\sqrt{n})$ 个单点。
- 询问单点的方法，首先把修改差分，于是第二棵树上的单点询问就变成了询问第一棵树的子树和，在第一棵树上把 dfs 序分成 $\sqrt{n}$ 个区间，然后维护一个 $\Theta(\sqrt{n}) - \Theta(1)$ 的数据结构即可。
- 这部分的时间复杂度是 $\Theta(n\sqrt{n})$ 。

QOJ9492. 树上简单求和

- 然后我们考虑处理关键点之间的询问，差分之后变为关键点到根的路径询问。

QOJ9492. 树上简单求和

- 然后我们考虑处理关键点之间的询问，差分之后变为关键点到根的路径询问。
- 注意到关键点只有 $\sqrt{n}$ 个，所以我们考虑用 $\Theta(n)$ 的时间计算一个关键点到第二棵树的根的所有询问信息。这个时候，我们把第二棵树上关键点到根的路径称为黑点，其它的称为白点。

QOJ9492. 树上简单求和

- 然后我们考虑处理关键点之间的询问，差分之后变为关键点到根的路径询问。
- 注意到关键点只有 $\sqrt{n}$ 个，所以我们考虑用 $\Theta(n)$ 的时间计算一个关键点到第二棵树的根的所有询问信息。这个时候，我们把第二棵树上关键点到根的路径称为黑点，其它的称为白点。
- 具体的，我们记录 s_i 表示第一棵树第 i 个点到根的路径上有多少个黑点，跟单点询问一样的差分，然后使用 $\Theta(\sqrt{n}) - \Theta(1)$ 的数据结构可以做到 $\Theta(n)$ 。

QOJ9492. 树上简单求和

- 然后我们考虑处理关键点之间的询问，差分之后变为关键点到根的路径询问。
- 注意到关键点只有 $\sqrt{n}$ 个，所以我们考虑用 $\Theta(n)$ 的时间计算一个关键点到第二棵树的根的所有询问信息。这个时候，我们把第二棵树上关键点到根的路径称为黑点，其它的称为白点。
- 具体的，我们记录 s_i 表示第一棵树第 i 个点到根的路径上有多少个黑点，跟单点询问一样的差分，然后使用 $\Theta(\sqrt{n}) - \Theta(1)$ 的数据结构可以做到 $\Theta(n)$ 。
- 总的时间复杂度是 $\Theta(n\sqrt{n})$ 。

LOJ4983.Impreza

- 给定一个 n 个点 m 条边的图。
- 问是否存在四个不同的点 a, b, c, d 使得 $(a, b), (a, c), (a, d), (b, c), (b, d), (c, d)$ 恰好有一条边不存在。
- 判断是否存在满足上述条件的四人组，如果存在，输出任意一组符合条件的即可。
- 数据范围： $n, m \leq 10^5$ 。

LOJ4983.Impreza

- 首先可以枚举所有的三元环 (a, b, c) ，记录 $mp[i][j]$ 表示可以与 i, j 组成三元环的点集，这里记录的是 *hash* 值

LOJ4983.Impreza

- 首先可以枚举所有的三元环 (a, b, c) ，记录 $mp[i][j]$ 表示可以与 i, j 组成三元环的点集，这里记录的是 *hash* 值
- 然后再枚举所有的三元环 (a, b, c) 考虑尝试找到一个点 d 使得 $(a, d), (b, d), (c, d)$ 恰好有两条边。

LOJ4983.Impreza

- 首先可以枚举所有的三元环 (a, b, c) ，记录 $mp[i][j]$ 表示可以与 i, j 组成三元环的点集，这里记录的是 *hash* 值
- 然后再枚举所有的三元环 (a, b, c) 考虑尝试找到一个点 d 使得 $(a, d), (b, d), (c, d)$ 恰好有两条边。
- 这就等价于判断 $(a, b, d), (a, c, d), (b, c, d)$ 恰好只有一个三元环。

LOJ4983.Impreza

- 首先可以枚举所有的三元环 (a, b, c) ，记录 $mp[i][j]$ 表示可以与 i, j 组成三元环的点集，这里记录的是 *hash* 值
- 然后再枚举所有的三元环 (a, b, c) 考虑尝试找到一个点 d 使得 $(a, d), (b, d), (c, d)$ 恰好有两条边。
- 这就等价于判断 $(a, b, d), (a, c, d), (b, c, d)$ 恰好只有一个三元环。
- 更进一步的，只需要判断 $(a, b, d), (a, c, d), (b, c, d)$ 存在有一组是三元环并且存在有一组不是三元环，那么剩下来的一组一定不是三元环，因为我们已知 (a, b, c) 是三元环。

LOJ4983.Impreza

- 首先可以枚举所有的三元环 (a, b, c) ，记录 $mp[i][j]$ 表示可以与 i, j 组成三元环的点集，这里记录的是 *hash* 值
- 然后再枚举所有的三元环 (a, b, c) 考虑尝试找到一个点 d 使得 $(a, d), (b, d), (c, d)$ 恰好有两条边。
- 这就等价于判断 $(a, b, d), (a, c, d), (b, c, d)$ 恰好只有一个三元环。
- 更进一步的，只需要判断 $(a, b, d), (a, c, d), (b, c, d)$ 存在有一组是三元环并且存在有一组不是三元环，那么剩下来的一组一定不是三元环，因为我们已知 (a, b, c) 是三元环。
- 所以判断一组存在一组不存在直接使用 *hash* 即可。

LOJ4983.Impreza

- 首先可以枚举所有的三元环 (a, b, c) ，记录 $mp[i][j]$ 表示可以与 i, j 组成三元环的点集，这里记录的是 *hash* 值
- 然后再枚举所有的三元环 (a, b, c) 考虑尝试找到一个点 d 使得 $(a, d), (b, d), (c, d)$ 恰好有两条边。
- 这就等价于判断 $(a, b, d), (a, c, d), (b, c, d)$ 恰好只有一个三元环。
- 更进一步的，只需要判断 $(a, b, d), (a, c, d), (b, c, d)$ 存在有一组是三元环并且存在有一组不是三元环，那么剩下来的一组一定不是三元环，因为我们已知 (a, b, c) 是三元环。
- 所以判断一组存在一组不存在直接使用 *hash* 即可。
- 最后考虑如果还原初始解，因为我们已经确定了三个点，所以直接从 $1 \sim n$ 枚举最后一个点，然后暴力 *check* 这四个点之间是不是恰好 5 条边。

LOJ4983.Impreza

- 首先可以枚举所有的三元环 (a, b, c) ，记录 $mp[i][j]$ 表示可以与 i, j 组成三元环的点集，这里记录的是 *hash* 值
- 然后再枚举所有的三元环 (a, b, c) 考虑尝试找到一个点 d 使得 $(a, d), (b, d), (c, d)$ 恰好有两条边。
- 这就等价于判断 $(a, b, d), (a, c, d), (b, c, d)$ 恰好只有一个三元环。
- 更进一步的，只需要判断 $(a, b, d), (a, c, d), (b, c, d)$ 存在有一组是三元环并且存在有一组不是三元环，那么剩下来的一组一定不是三元环，因为我们已知 (a, b, c) 是三元环。
- 所以判断一组存在一组不存在直接使用 *hash* 即可。
- 最后考虑如果还原初始解，因为我们已经确定了三个点，所以直接从 $1 \sim n$ 枚举最后一个点，然后暴力 *check* 这四个点之间是不是恰好 5 条边。
- 时间复杂度 $\Theta(n\sqrt{n})$ 。

P4747 [CERC2017] Intrinsic Interval

- 对于 $1 \sim n$ 的排列 π ，若一个子串 $\pi[a \dots b]$ 排序之后是连续正整数，则称 $\pi[a, b]$ 是一个“区间”。
- 一个子串的本征区间是包含该子串的最短区间，这里包含指：若区间 $\pi[x \dots y]$ 的本征区间为 $\pi[a \dots b]$ ，则必定有 $a \leq x \leq y \leq b$ 。
- 给定长度为 n 的排列和 q 次询问，每次询问 x, y ，求出 $\pi[x \dots y]$ 的本征区间。
- 数据范围： $n, m \leq 10^5$ 。

P4747 [CERC2017] Intrinsic Interval

- 考虑这道题的本征区间，就等价于控制了一个区间的极差。

P4747 [CERC2017] Intrinsic Interval

- 考虑这道题的本征区间，就等价于控制了一个区间的极差。
- 具体的，考虑对于一个区间 $\pi[l \dots r]$ ，如果这个区间是一个本征区间当且仅当 $\max - \min = r - l$ 。

P4747 [CERC2017] Intrinsic Interval

- 考虑这道题的本征区间，就等价于控制了一个区间的极差。
- 具体的，考虑对于一个区间 $\pi[l \dots r]$ ，如果这个区间是一个本征区间当且仅当 $\max - \min = r - l$ 。
- 同时对于任意的区间 $\pi[l, r]$ 都有 $\max - \min \geq r - l$ 。

P4747 [CERC2017] Intrinsic Interval

- 考虑这道题的本征区间，就等价于控制了一个区间的极差。
- 具体的，考虑对于一个区间 $\pi[l \dots r]$ ，如果这个区间是一个本征区间当且仅当 $\max - \min = r - l$ 。
- 同时对于任意的区间 $\pi[l, r]$ 都有 $\max - \min \geq r - l$ 。
- 于是可以尝试维护一棵线段树，对于每个点存储区间 $\Delta = \max - \min - (r - l)$ 的最小值和最小值所对应的最大位置。

P4747 [CERC2017] Intrinsic Interval

- 考虑这道题的本征区间，就等价于控制了一个区间的极差。
- 具体的，考虑对于一个区间 $\pi[l \dots r]$ ，如果这个区间是一个本征区间当且仅当 $\max - \min = r - l$ 。
- 同时对于任意的区间 $\pi[l, r]$ 都有 $\max - \min \geq r - l$ 。
- 于是可以尝试维护一棵线段树，对于每个点存储区间 $\Delta = \max - \min - (r - l)$ 的最小值和最小值所对应的最大位置。
- 考虑 r 从小往大扫描，每次 $r \rightarrow r + 1$ ，所以要给所有区间 -1 。同时我们需要检查 $\max, \min$ 的变化。

P4747 [CERC2017] Intrinsic Interval

- 考虑这道题的本征区间，就等价于控制了一个区间的极差。
- 具体的，考虑对于一个区间 $\pi[l \dots r]$ ，如果这个区间是一个本征区间当且仅当 $\max - \min = r - l$ 。
- 同时对于任意的区间 $\pi[l, r]$ 都有 $\max - \min \geq r - l$ 。
- 于是可以尝试维护一棵线段树，对于每个点存储区间 $\Delta = \max - \min - (r - l)$ 的最小值和最小值所对应的最大位置。
- 考虑 r 从小往大扫描，每次 $r \rightarrow r + 1$ ，所以要给所有区间 -1 。同时我们需要检查 $\max, \min$ 的变化。
- 对于这道题，我们并不适合进行区间赋值的操作。但我们考虑维护出 $\max, \min$ 的单调栈，单调栈发生变化的时候，动态维护 Δ 的变化即可。

P4747 [CERC2017] Intrinsic Interval

- 考虑查询，因为 r 是从小往大扫的。对于询问 $[x, y]$ ，首先考虑一个暴力的做法，就是对于所有的 $r \geq y$ 全部找到比 x 小的最大的合法的 l' ，然后用 $\pi[l' \dots r]$ 更新答案。

P4747 [CERC2017] Intrinsic Interval

- 考虑查询，因为 r 是从小往大扫的。对于询问 $[x, y]$ ，首先考虑一个暴力的做法，就是对于所有的 $r \geq y$ 全部找到比 x 小的最大的合法的 l' ，然后用 $\pi[l' \dots r]$ 更新答案。
- 进一步对于这个算法进行优化，首先可以分析，在扫描的过程中，遇到了第一个合法的解就可以停止扫描。

P4747 [CERC2017] Intrinsic Interval

- 考虑查询，因为 r 是从小往大扫的。对于询问 $[x, y]$ ，首先考虑一个暴力的做法，就是对于所有的 $r \geq y$ 全部找到比 x 小的最大的合法的 l' ，然后用 $\pi[l' \dots r]$ 更新答案。
- 进一步对于这个算法进行优化，首先可以分析，在扫描的过程当中，遇到了第一个合法的解就可以停止扫描。
- 也就是说，如果存在两个包含 $[x, y]$ 的区间 $[l_1, r_1], [l_2, r_2]$ 有 $l_1 \leq l_2, r_1 \leq r_2$ ，也即 $l_1 \leq l_2 \leq x \leq y \leq r_1 \leq r_2$ ，那么我们不需要考虑 $[l_1, r_1], [l_2, r_2]$ ，因为 $[l_2, r_1]$ 一定是一组更优的解。

P4747 [CERC2017] Intrinsic Interval

- 考虑查询，因为 r 是从小往大扫的。对于询问 $[x, y]$ ，首先考虑一个暴力的做法，就是对于所有的 $r \geq y$ 全部找到比 x 小的最大的合法的 l' ，然后用 $\pi[l' \dots r]$ 更新答案。
- 进一步对于这个算法进行优化，首先可以分析，在扫描的过程中，遇到了第一个合法的解就可以停止扫描。
- 也就是说，如果存在两个包含 $[x, y]$ 的区间 $[l_1, r_1], [l_2, r_2]$ 有 $l_1 \leq l_2, r_1 \leq r_2$ ，也即 $l_1 \leq l_2 \leq x \leq y \leq r_1 \leq r_2$ ，那么我們不需要考虑 $[l_1, r_1], [l_2, r_2]$ ，因为 $[l_2, r_1]$ 一定是一组更优的解。
- 知道了上述结论，可以考虑当扫到 $r = y$ 的时候插入 x 进入一个询问队列。在任意一个时刻， x 越大都越有可能找到一组合法的解，于是对于这个优先队列，我们每次判断 x 最大的是否存在一组解，如果不存在就不再判断剩下的数，如果存在就弹出最大值之后考虑次大值，这样我们就不会造成额外的浪费。

P4747 [CERC2017] Intrinsic Interval

- 考虑查询，因为 r 是从小往大扫的。对于询问 $[x, y]$ ，首先考虑一个暴力的做法，就是对于所有的 $r \geq y$ 全部找到比 x 小的最大的合法的 l' ，然后用 $\pi[l' \dots r]$ 更新答案。
- 进一步对于这个算法进行优化，首先可以分析，在扫描的过程当中，遇到了第一个合法的解就可以停止扫描。
- 也就是说，如果存在两个包含 $[x, y]$ 的区间 $[l_1, r_1], [l_2, r_2]$ 有 $l_1 \leq l_2, r_1 \leq r_2$ ，也即 $l_1 \leq l_2 \leq x \leq y \leq r_1 \leq r_2$ ，那么我們不需要考虑 $[l_1, r_1], [l_2, r_2]$ ，因为 $[l_2, r_1]$ 一定是一组更优的解。
- 知道了上述结论，可以考虑当扫到 $r = y$ 的时候插入 x 进入一个询问队列。在任意一个时刻， x 越大都越有可能找到一组合法的解，于是对于这个优先队列，我们每次判断 x 最大的是否存在一组解，如果不存在就不再判断剩下的数，如果存在就弹出最大值之后考虑次大值，这样我们就不会造成额外的浪费。
- 时间复杂度 $\Theta(n \log n)$ 。

P8528 [Ynoi2003] 铃原露露

- 给定一棵有根树，顶点编号为 $1, 2, \dots, n$ ，对 $2 \leq i \leq n$ ，有 f_i 为 i 的父亲。
- $a_1, a_2, \dots, a_n$ 是 $1, \dots, n$ 的排列。
- 有 m 次询问，每次询问给定 l, r ，询问有多少个二元组 L, R 满足 $l \leq L \leq R \leq r$ ，且对任意 $L \leq a_x \leq a_y \leq R$ ，有 x, y 在树上的最近公共祖先 z 满足 $L \leq a_z \leq R$ 。
- $n, m \leq 2 \times 10^5, 1 \leq f_i \leq i - 1$ 。

P8528 [Ynoi2003] 铃原露露

- 考虑点权为 $u, v (u < v)$ 的最近公共祖先为 w 。然后分类讨论 u, v, w 的大小关系。

P8528 [Ynoi2003] 铃原露露

- 考虑点权为 $u, v (u < v)$ 的最近公共祖先为 w 。然后分类讨论 u, v, w 的大小关系。
- 若 $u < w < v$ ，没有影响。

P8528 [Ynoi2003] 铃原露露

- 考虑点权为 $u, v (u < v)$ 的最近公共祖先为 w 。然后分类讨论 u, v, w 的大小关系。
- 若 $u < w < v$ ，没有影响。
- 若 $u < v < w$ ，则 $l \leq u, r \in [v, w)$ 的区间不合法。

P8528 [Ynoi2003] 铃原露露

- 考虑点权为 $u, v (u < v)$ 的最近公共祖先为 w 。然后分类讨论 u, v, w 的大小关系。
- 若 $u < w < v$ ，没有影响。
- 若 $u < v < w$ ，则 $l \leq u, r \in [v, w)$ 的区间不合法。
- 若 $w < u < v$ ，则 $l \in (w, u], r \geq v$ 的区间不合法。

P8528 [Ynoi2003] 铃原露露

- 考虑点权为 $u, v (u < v)$ 的最近公共祖先为 w 。然后分类讨论 u, v, w 的大小关系。
- 若 $u < w < v$ ，没有影响。
- 若 $u < v < w$ ，则 $l \leq u, r \in [v, w)$ 的区间不合法。
- 若 $w < u < v$ ，则 $l \in (w, u], r \geq v$ 的区间不合法。
- 考虑树上启发式合并，每个点插入 **set** 的时候只考虑前驱和后继，于是有 $\Theta(n \log n)$ 对支配对。

P8528 [Ynoi2003] 铃原露露

- 考虑点权为 $u, v (u < v)$ 的最近公共祖先为 w 。然后分类讨论 u, v, w 的大小关系。
- 若 $u < w < v$ ，没有影响。
- 若 $u < v < w$ ，则 $l \leq u, r \in [v, w)$ 的区间不合法。
- 若 $w < u < v$ ，则 $l \in (w, u], r \geq v$ 的区间不合法。
- 考虑树上启发式合并，每个点插入 set 的时候只考虑前驱和后继，于是有 $\Theta(n \log n)$ 对支配对。
- 询问类似询问矩形合法点对个数，线段树维护，扫描线维护矩形面积并即可。

P8528 [Ynoi2003] 铃原露露

- 考虑点权为 $u, v (u < v)$ 的最近公共祖先为 w 。然后分类讨论 u, v, w 的大小关系。
- 若 $u < w < v$ ，没有影响。
- 若 $u < v < w$ ，则 $l \leq u, r \in [v, w)$ 的区间不合法。
- 若 $w < u < v$ ，则 $l \in (w, u], r \geq v$ 的区间不合法。
- 考虑树上启发式合并，每个点插入 set 的时候只考虑前驱和后继，于是有 $\Theta(n \log n)$ 对支配对。
- 询问类似询问矩形合法点对个数，线段树维护，扫描线维护矩形面积并即可。
- 时间复杂度 $\Theta(n \log^2 n)$ 。

- ① 闲话
- ② Prob1-4
- ③ Prob5-8
- ④ Prob9-11

P6816 [PA 2009] Quasi-template

- 定义一个串 s 能匹配 S 当且仅当 s 能可超出头尾地覆盖 S 且长度不超过 S ，且 s 必须是 S 的子串。
- 例如，串 $aabaa$ 可以匹配 $aaaabaabaaaba$ 。
- 给定 S ，求不同的 s 个数以及长度最短的 s ，如果有多解，输出字典序最小解。
- $|S| \leq 2 \times 10^5$ 。

P6816 [PA 2009] Quasi-template

- 首先考虑我们判断一个字符串 s 是否能匹配：

P6816 [PA 2009] Quasi-template

- 首先考虑我们判断一个字符串 s 是否能匹配：
- 求出 s 所有的出现位置，判断中间的是否能填满，这一部分就是比较 s 的长度和出现位置的极差。

P6816 [PA 2009] Quasi-template

- 首先考虑我们判断一个字符串 s 是否能匹配：
- 求出 s 所有的出现位置，判断中间的是否能填满，这一部分就是比较 s 的长度和出现位置的极差。
- 对于头尾，判断开头部分能不能被一个后缀覆盖，同理，判断后缀能不能被一个前缀覆盖。

P6816 [PA 2009] Quasi-template

- 首先考虑我们判断一个字符串 s 是否能匹配：
- 求出 s 所有的出现位置，判断中间的是否能填满，这一部分就是比较 s 的长度和出现位置的极差。
- 对于头尾，判断开头部分能不能被一个后缀覆盖，同理，判断后缀能不能被一个前缀覆盖。
- 首先考虑计数有多少个合法的子串。

P6816 [PA 2009] Quasi-template

- 首先考虑我们判断一个字符串 s 是否能匹配：
- 求出 s 所有的出现位置，判断中间的是否能填满，这一部分就是比较 s 的长度和出现位置的极差。
- 对于头尾，判断开头部分能不能被一个后缀覆盖，同理，判断后缀能不能被一个前缀覆盖。
- 首先考虑计数有多少个合法的子串。
- 建出后缀自动机，然后用线段树合并维护出现位置的极差，并且维护最早出现和最后出现 $endpos$ 位置，并且有属于这个等价类的字符串的长度区间。

P6816 [PA 2009] Quasi-template

- 首先考虑我们判断一个字符串 s 是否能匹配：
- 求出 s 所有的出现位置，判断中间的是否能填满，这一部分就是比较 s 的长度和出现位置的极差。
- 对于头尾，判断开头部分能不能被一个后缀覆盖，同理，判断后缀能不能被一个前缀覆盖。
- 首先考虑计数有多少个合法的子串。
- 建出后缀自动机，然后用线段树合并维护出现位置的极差，并且维护最早出现和最后出现 $endpos$ 位置，并且有属于这个等价类的字符串的长度区间。
- 假设字符串的长度区间是 $[l, r]$ ，出现位置极差为 v ，只需要 $l = \max(l, v)$ 即可。

P6816 [PA 2009] Quasi-template

- 考虑首尾的限制。

P6816 [PA 2009] Quasi-template

- 考虑首尾的限制。
- 设 *endpos* 的最靠前的位置为 L ，那么我们需要额外覆盖 $S[1, L - len]$ 的部分，这里的 len 指的是字符串 s 长度。

P6816 [PA 2009] Quasi-template

- 考虑首尾的限制。
- 设 $endpos$ 的最靠前的位置为 L ，那么我们需要额外覆盖 $S[1, L - len]$ 的部分，这里的 len 指的是字符串 s 长度。
- 于是我们要求，存在 $x \geq L - len$ 使得 $S[1, x]$ 是 s 的一个后缀。又因为 $S[L - len + 1, L] = s$ ，所以 $S[1, x]$ 是 $S[1, L]$ 的 *border*。设 f_i 表示 $S[1, i]$ 的 *border* 长度。

P6816 [PA 2009] Quasi-template

- 考虑首尾的限制。
- 设 *endpos* 的最靠前的位置为 L ，那么我们需要额外覆盖 $S[1, L - len]$ 的部分，这里的 *len* 指的是字符串 s 长度。
- 于是我们要求，存在 $x \geq L - len$ 使得 $S[1, x]$ 是 s 的一个后缀。又因为 $S[L - len + 1, L] = s$ ，所以 $S[1, x]$ 是 $S[1, L]$ 的 *border*。设 f_i 表示 $S[1, i]$ 的 *border* 长度。
- 于是我们的判断条件就变成了： $len \geq \frac{1}{2}$ 和 $len \geq f_i$ 。求出所有前缀的 *border* 长度即可。

P6816 [PA 2009] Quasi-template

- 考虑首尾的限制。
- 设 *endpos* 的最靠前的位置为 L ，那么我们需要额外覆盖 $S[1, L - len]$ 的部分，这里的 *len* 指的是字符串 s 长度。
- 于是我们要求，存在 $x \geq L - len$ 使得 $S[1, x]$ 是 s 的一个后缀。又因为 $S[L - len + 1, L] = s$ ，所以 $S[1, x]$ 是 $S[1, L]$ 的 *border*。设 f_i 表示 $S[1, i]$ 的 *border* 长度。
- 于是我们的判断条件就变成了： $len \geq \frac{1}{2}$ 和 $len \geq f_i$ 。求出所有前缀的 *border* 长度即可。
- 后缀是类似的，设 $g[i]$ 表示 $S[i, n]$ 的 *border* 长度。只不过变成了 $g[x] \geq n - R$ ， R 是 *endpos* 最靠后的位置。

P6816 [PA 2009] Quasi-template

- 考虑首尾的限制。
- 设 $endpos$ 的最靠前的位置为 L ，那么我们需要额外覆盖 $S[1, L - len]$ 的部分，这里的 len 指的是字符串 s 长度。
- 于是我们要求，存在 $x \geq L - len$ 使得 $S[1, x]$ 是 s 的一个后缀。又因为 $S[L - len + 1, L] = s$ ，所以 $S[1, x]$ 是 $S[1, L]$ 的 *border*。设 f_i 表示 $S[1, i]$ 的 *border* 长度。
- 于是我们的判断条件就变成了： $len \geq \frac{1}{2}$ 和 $len \geq f_i$ 。求出所有前缀的 *border* 长度即可。
- 后缀是类似的，设 $g[i]$ 表示 $S[i, n]$ 的 *border* 长度。只不过变成了 $g[x] \geq n - R$ ， R 是 $endpos$ 最靠后的位置。
- 于是我们考虑就相当于数一个区间内有多少个 $g[x] \geq n - R$ ，用主席树即可。

P6816 [PA 2009] Quasi-template

- 最后考虑计算字典序最小解。

P6816 [PA 2009] Quasi-template

- 最后考虑计算字典序最小解。
- 实时维护最优解。

P6816 [PA 2009] Quasi-template

- 最后考虑计算字典序最小解。
- 实时维护最优解。
- 每次更新答案的时候比较一下最优解是否被更新。

P6816 [PA 2009] Quasi-template

- 最后考虑计算字典序最小解。
- 实时维护最优解。
- 每次更新答案的时候比较一下最优解是否被更新。
- 这里直接用二分 *lcp* 的办法容易比较两个串的字典序。

P6816 [PA 2009] Quasi-template

- 最后考虑计算字典序最小解。
- 实时维护最优解。
- 每次更新答案的时候比较一下最优解是否被更新。
- 这里直接用二分 lcp 的办法容易比较两个串的字典序。
- 时间复杂度 $\Theta(n \log n)$ 。

QOJ8035.Call Me Call Me

- 给定 n 个点，第 i 个点能够变成黑色当且仅当编号为 $[l_i, r_i]$ 的点中至少有 k 个黑色。请求出最多能够被染黑的点数。
- 数据范围： $1 \leq n \leq 4 \times 10^5$ ，时间限制 15s。
- 提示：思考一下不带根号的做法（虽然这道题跑得很快的根号能过）。

QOJ8035.Call Me Call Me

- 考虑在线段树当中使用减半报警器。

QOJ8035.Call Me Call Me

- 考虑在线段树当中使用减半报警器。
- 对于线段树中的一个结点，如果询问区间 $[l, r]$ 跨过了 mid 的时候，维护 $[l, mid], (mid, r]$ 两个报警器。

QOJ8035.Call Me Call Me

- 考虑在线段树当中使用减半报警器。
- 对于线段树中的一个结点，如果询问区间 $[l, r]$ 跨过了 mid 的时候，维护 $[l, mid], (mid, r]$ 两个报警器。
- 具体的，在每个线段树结点内部新开一个线段树表示该点报警器的情况。

QOJ8035.Call Me Call Me

- 考虑在线段树当中使用减半报警器。
- 对于线段树中的一个结点，如果询问区间 $[l, r]$ 跨过了 mid 的时候，维护 $[l, mid], (mid, r]$ 两个报警器。
- 具体的，在每个线段树结点内部新开一个线段树表示该点报警器的情况。
- 当一个节点被染黑的时候，相当于对外层线段树遍历到的节点的内层线段树进行区间减的操作。

QOJ8035.Call Me Call Me

- 考虑在线段树当中使用减半报警器。
- 对于线段树中的一个结点，如果询问区间 $[l, r]$ 跨过了 mid 的时候，维护 $[l, mid], (mid, r]$ 两个报警器。
- 具体的，在每个线段树结点内部新开一个线段树表示该点报警器的情况。
- 当一个节点被染黑的时候，相当于对外层线段树遍历到的节点的内层线段树进行区间减的操作。
- 然后对于每个内层的线段树，维护区间最小值，以便于监视是否有报警器归零。

QOJ8035.Call Me Call Me

- 考虑在线段树当中使用减半报警器。
- 对于线段树中的一个结点，如果询问区间 $[l, r]$ 跨过了 mid 的时候，维护 $[l, mid], (mid, r]$ 两个报警器。
- 具体的，在每个线段树结点内部新开一个线段树表示该点报警器的情况。
- 当一个节点被染黑的时候，相当于对外层线段树遍历到的节点的内层线段树进行区间减的操作。
- 然后对于每个内层的线段树，维护区间最小值，以便于监视是否有报警器归零。
- 时间复杂度 $\Theta(n \log^2 n)$ 。

P5048 [Ynoi2019 模拟赛] Yuno loves sqrt technology III

- 给定一个长度为 n 的序列 a 。
- m 次询问，每次查询一个区间的众数出现次数。
- 强制在线。
- 数据范围： $n, m, a_i \leq 5 \times 10^5$ ，2 秒，线性空间。

P5048 [Ynoi2019 模拟赛] Yuno loves sqrt technology III

- 考虑按照 $\sqrt{n}$ 分块，设 $f[L][R]$ 表示第 L 个块到第 R 个块所构成的区间众数出现次数。

P5048 [Ynoi2019 模拟赛] Yuno loves sqrt technology III

- 考虑按照 $\sqrt{n}$ 分块，设 $f[L][R]$ 表示第 L 个块到第 R 个块所构成的区间众数出现次数。
- 预处理的方式考虑枚举 L 然后暴力扫整个序列，时间复杂度 $\Theta(n\sqrt{n})$ 。

P5048 [Ynoi2019 模拟赛] Yuno loves sqrt technology III

- 考虑按照 $\sqrt{n}$ 分块，设 $f[L][R]$ 表示第 L 个块到第 R 个块所构成的区间众数出现次数。
- 预处理的方式考虑枚举 L 然后暴力扫整个序列，时间复杂度 $\Theta(n\sqrt{n})$ 。
- 对于询问，可以直接知道整块的答案，设为 *ans*，然后把散块的答案加进来。

P5048 [Ynoi2019 模拟赛] Yuno loves sqrt technology III

- 考虑按照 $\sqrt{n}$ 分块，设 $f[L][R]$ 表示第 L 个块到第 R 个块所构成的区间众数出现次数。
- 预处理的方式考虑枚举 L 然后暴力扫整个序列，时间复杂度 $\Theta(n\sqrt{n})$ 。
- 对于询问，可以直接知道整块的答案，设为 ans ，然后把散块的答案加进来。
- 因为散块的大小，答案一定不会超过 $ans + 2\sqrt{n}$ 。我们考虑对于散块中的元素判断出现次数能不能达到 $ans + 1$ 。

P5048 [Ynoi2019 模拟赛] Yuno loves sqrt technology III

- 考虑按照 $\sqrt{n}$ 分块，设 $f[L][R]$ 表示第 L 个块到第 R 个块所构成的区间众数出现次数。
- 预处理的方式考虑枚举 L 然后暴力扫整个序列，时间复杂度 $\Theta(n\sqrt{n})$ 。
- 对于询问，可以直接知道整块的答案，设为 ans ，然后把散块的答案加进来。
- 因为散块的大小，答案一定不会超过 $ans + 2\sqrt{n}$ 。我们考虑对于散块中的元素判断出现次数能不能达到 $ans + 1$ 。
- 具体的，可以把相同的数存进一个 **vector**。

P5048 [Ynoi2019 模拟赛] Yuno loves sqrt technology III

- 考虑按照 $\sqrt{n}$ 分块，设 $f[L][R]$ 表示第 L 个块到第 R 个块所构成的区间众数出现次数。
- 预处理的方式考虑枚举 L 然后暴力扫整个序列，时间复杂度 $\Theta(n\sqrt{n})$ 。
- 对于询问，可以直接知道整块的答案，设为 ans ，然后把散块的答案加进来。
- 因为散块的大小，答案一定不会超过 $ans + 2\sqrt{n}$ 。我们考虑对于散块中的元素判断出现次数能不能达到 $ans + 1$ 。
- 具体的，可以把相同的数存进一个 $vector$ 。
- 对于开头的散块部分，第 x 个位置就检查在 $vector$ 当中 x 往后 $ans + 1$ 个（如果存在）是否仍然在区间当中。结尾部分的散块处理方式类似。

P5048 [Ynoi2019 模拟赛] Yuno loves sqrt technology III

- 考虑按照 $\sqrt{n}$ 分块，设 $f[L][R]$ 表示第 L 个块到第 R 个块所构成的区间众数出现次数。
- 预处理的方式考虑枚举 L 然后暴力扫整个序列，时间复杂度 $\Theta(n\sqrt{n})$ 。
- 对于询问，可以直接知道整块的答案，设为 ans ，然后把散块的答案加进来。
- 因为散块的大小，答案一定不会超过 $ans + 2\sqrt{n}$ 。我们考虑对于散块中的元素判断出现次数能不能达到 $ans + 1$ 。
- 具体的，可以把相同的数存进一个 $vector$ 。
- 对于开头的散块部分，第 x 个位置就检查在 $vector$ 当中 x 往后 $ans + 1$ 个（如果存在）是否仍然在区间当中。结尾部分的散块处理方式类似。
- 时间复杂度 $\Theta(n\sqrt{n})$ 。

QOJ3047. Wind of Change

- 给定两棵有 n 个点的树 T_1, T_2 , 边有边权。
- 对于每个结点 x 求出 $\min_{x \neq y} dis1(x, y) + dis2(x, y)$ 。
- 其中 $dis_i(x, y)$ 表示在第 i 棵树上 x, y 之间最短路径的边权和。
- 数据范围: $n \leq 2 \times 10^5, w_i \leq 10^9$ 。

QOJ3047. Wind of Change

- 考虑对于两棵树都点分治。

QOJ3047. Wind of Change

- 考虑对于两棵树都点分治。
- 那么我们将距离转化为两个点到两棵树点分树的 lca 距离和。

QOJ3047. Wind of Change

- 考虑对于两棵树都点分治。
- 那么我们将距离转化为两个点到两棵树点分树的 lca 距离和。
- 固定第一棵树的 lca 设为点 p_1 ，那么可以看成 p_1 子树内的点在第二棵树上的距离和再加一个权值。

QOJ3047. Wind of Change

- 考虑对于两棵树都点分治。
- 那么我们将距离转化为两个点到两棵树点分树的 lca 距离和。
- 固定第一棵树的 lca 设为点 p_1 ，那么可以看成 p_1 子树内的点在第二棵树上的距离和再加一个权值。
- 然后枚举点 x ，再枚举第二棵树的 lca 设为 p_2 。

QOJ3047. Wind of Change

- 考虑对于两棵树都点分治。
- 那么我们将距离转化为两个点到两棵树点分树的 lca 距离和。
- 固定第一棵树的 lca 设为点 p_1 ，那么可以看成 p_1 子树内的点在第二棵树上的距离和再加一个权值。
- 然后枚举点 x ，再枚举第二棵树的 lca 设为 p_2 。
- 如果固定了两个点在两个点分树的最近公共祖先，那么距离就可以完全被拆成两个独立的部分。

QOJ3047. Wind of Change

- 考虑对于两棵树都点分治。
- 那么我们将距离转化为两个点到两棵树点分树的 lca 距离和。
- 固定第一棵树的 lca 设为点 p_1 ，那么可以看成 p_1 子树内的点在第二棵树上的距离和再加一个权值。
- 然后枚举点 x ，再枚举第二棵树的 lca 设为 p_2 。
- 如果固定了两个点在两个点分树的最近公共祖先，那么距离就可以完全被拆成两个独立的部分。
- 于是我们可以直接维护最小值和最小值所对应的点编号，在枚举 p_2 的时候更新答案即可。

QOJ3047. Wind of Change

- 考虑对于两棵树都点分治。
- 那么我们将距离转化为两个点到两棵树点分树的 lca 距离和。
- 固定第一棵树的 lca 设为点 p_1 ，那么可以看成 p_1 子树内的点在第二棵树上的距离和再加一个权值。
- 然后枚举点 x ，再枚举第二棵树的 lca 设为 p_2 。
- 如果固定了两个点在两个点分树的最近公共祖先，那么距离就可以完全被拆成两个独立的部分。
- 于是我们可以直接维护最小值和最小值所对应的点编号，在枚举 p_2 的时候更新答案即可。
- 因为点分树上一个点最多只有 $\Theta(\log n)$ 个祖先，时间复杂度 $\Theta(n \log^2 n)$ 。

- ① 闲话
- ② Prob1-4
- ③ Prob5-8
- ④ Prob9-11

QOJ1163. Another Tree Queries Problem

- 给定一个大小为 n 个点的树，每个点有一个点权 a_i ，初始所有点的点权都是 0。
- 一共有 m 次操作，操作分为如下三种：
- “1 $u\ v$ ”：以 u 为根， v 的子树内点权加一。
- “2 $u\ v$ ”：(u, v) 最短路径上的点权加一。
- “3 v ”：输出 $\sum_{i=1}^n \text{dist}(v, i) \times a_i$ ，其中 $\text{dist}(v, i)$ 是 (v, i) 最短路径的边数。
- 数据范围： $n, m \leq 2 \times 10^5$ 。

QOJ1163. Another Tree Queries Problem

- 考虑处理询问，把 $dist(v, i)$ 拆成 $dep_v + dep_i - 2 \times dep_{lca}$ 。

QOJ1163. Another Tree Queries Problem

- 考虑处理询问，把 $dist(v, i)$ 拆成 $dep_v + dep_i - 2 \times dep_{lca}$ 。
- 并且 $\sum_{i=1}^n dep_i \times a_i$ 和 $\sum_{i=1}^n dep_v \times a_i$ 都是好算的，只需要考虑 $\sum_{i=1}^n dep_{lca(v,i)} \times a_i$ 。

QOJ1163. Another Tree Queries Problem

- 考虑处理询问，把 $dist(v, i)$ 拆成 $dep_v + dep_i - 2 \times dep_{lca}$ 。
- 并且 $\sum_{i=1}^n dep_i \times a_i$ 和 $\sum_{i=1}^n dep_v \times a_i$ 都是好算的，只需要考虑 $\sum_{i=1}^n dep_{lca(v,i)} \times a_i$ 。
- 对于 $dep_{lca(v,i)}$ 的处理，考虑每次进行 a_i 加操作的时候，把 i 到根的路径全部进行加操作，然后询问的时候直接询问 v 到根的路径和。

QOJ1163. Another Tree Queries Problem

- 考虑处理询问，把 $dist(v, i)$ 拆成 $dep_v + dep_i - 2 \times dep_{lca}$ 。
- 并且 $\sum_{i=1}^n dep_i \times a_i$ 和 $\sum_{i=1}^n dep_v \times a_i$ 都是好算的，只需要考虑 $\sum_{i=1}^n dep_{lca(v,i)} \times a_i$ 。
- 对于 $dep_{lca(v,i)}$ 的处理，考虑每次进行 a_i 加操作的时候，把 i 到根的路径全部进行加操作，然后询问的时候直接询问 v 到根的路径和。
- 记转化后的每个点权值为 s_i 。

QOJ1163. Another Tree Queries Problem

- 考虑处理询问, 把 $dist(v, i)$ 拆成 $dep_v + dep_i - 2 \times dep_{lca}$ 。
- 并且 $\sum_{i=1}^n dep_i \times a_i$ 和 $\sum_{i=1}^n dep_v \times a_i$ 都是好算的, 只需要考虑 $\sum_{i=1}^n dep_{lca(v, i)} \times a_i$ 。
- 对于 $dep_{lca(v, i)}$ 的处理, 考虑每次进行 a_i 加操作的时候, 把 i 到根的路径全部进行加操作, 然后询问的时候直接询问 v 到根的路径和。
- 记转化后的每个点权值为 s_i 。
- 注意上述的 $lca(v, i)$ 都是以 1 为根的情况下考虑的最近公共祖先。

QOJ1163. Another Tree Queries Problem

- 考虑处理询问, 把 $dist(v, i)$ 拆成 $dep_v + dep_i - 2 \times dep_{lca}$ 。
- 并且 $\sum_{i=1}^n dep_i \times a_i$ 和 $\sum_{i=1}^n dep_v \times a_i$ 都是好算的, 只需要考虑 $\sum_{i=1}^n dep_{lca(v, i)} \times a_i$ 。
- 对于 $dep_{lca(v, i)}$ 的处理, 考虑每次进行 a_i 加操作的时候, 把 i 到根的路径全部进行加操作, 然后询问的时候直接询问 v 到根的路径和。
- 记转化后的每个点权值为 s_i 。
- 注意上述的 $lca(v, i)$ 都是以 1 为根的情况下考虑的最近公共祖先。
- 为了维护操作, 对于原树以 1 为根进行重链剖分。

QOJ1163. Another Tree Queries Problem

- 首先看第一个操作 $(1, u, v)$ 。

QOJ1163. Another Tree Queries Problem

- 首先看第一个操作 $(1, u, v)$ 。
- 那么这种情况下，操作其实就分为两种，一种是子树内的点权加一，一种是子树外的点权加一。我们只考虑字数内的点权加一，因为子树外点权加等于整棵树的点权加并且字数内的点权减。

QOJ1163. Another Tree Queries Problem

- 首先看第一个操作 $(1, u, v)$ 。
- 那么这种情况下，操作其实就分为两种，一种是子树内的点权加一，一种是子树外的点权加一。我们只考虑字数内的点权加一，因为子树外点权加等于整棵树的点权加并且字数内的点权减。
- 考虑这种情况下 s_i 的变化。

QOJ1163. Another Tree Queries Problem

- 首先看第一个操作 $(1, u, v)$ 。
- 那么这种情况下，操作其实就分为两种，一种是子树内的点权加一，一种是子树外的点权加一。我们只考虑字数内的点权加一，因为子树外点权加等于整棵树的点权加并且字数内的点权减。
- 考虑这种情况下 s_i 的变化。
- 对于 v 到根的路径上的点， $s_x += siz_v$ ，即加上了 v 子树大小，这是一个树上链加操作。

QOJ1163. Another Tree Queries Problem

- 首先看第一个操作 $(1, u, v)$ 。
- 那么这种情况下，操作其实就分为两种，一种是子树内的点权加一，一种是子树外的点权加一。我们只考虑字数内的点权加一，因为子树外点权加等于整棵树的点权加并且字数内的点权减。
- 考虑这种情况下 s_i 的变化。
- 对于 v 到根的路径上的点， $s_x += siz_v$ ，即加上了 v 子树大小，这是一个树上链加操作。
- 对于 v 子树内的点 x ， $s_x += dep_x - dep_v + 1$ 。拆成 $s_x += dep_x$ 和 $s_x -= dep_v - 1$ ，即分别为子树减对应点的深度与子树减一个固定值。

QOJ1163. Another Tree Queries Problem

- 首先看第一个操作 $(1, u, v)$ 。
- 那么这种情况下，操作其实就分为两种，一种是子树内的点权加一，一种是子树外的点权加一。我们只考虑字数内的点权加一，因为子树外点权加等于整棵树的点权加并且字数内的点权减。
- 考虑这种情况下 s_i 的变化。
- 对于 v 到根的路径上的点， $s_x += siz_v$ ，即加上了 v 子树大小，这是一个树上链加操作。
- 对于 v 子树内的点 x ， $s_x += dep_x - dep_v + 1$ 。拆成 $s_x += dep_x$ 和 $s_x -= dep_v - 1$ ，即分别为子树减对应点的深度与子树减一个固定值。
- 上述的操作都可以在重链剖分之后用线段树实现。

QOJ1163. Another Tree Queries Problem

- 然后第二个操作 $(2, u, v)$ 。

QOJ1163. Another Tree Queries Problem

- 然后第二个操作 $(2, u, v)$ 。
- 同理也可以拆成 lca 到根路径上的点和 (u, v) 最短路径上的点。

QOJ1163. Another Tree Queries Problem

- 然后第二个操作 $(2, u, v)$ 。
- 同理也可以拆成 lca 到根路径上的点和 (u, v) 最短路径上的点。
- 同样的前面的部分也是链加，第二种情况拆开后是链加对应的深度和链加固定数。

QOJ1163. Another Tree Queries Problem

- 然后第二个操作 $(2, u, v)$ 。
- 同理也可以拆成 lca 到根路径上的点和 (u, v) 最短路径上的点。
- 同样的前面的部分也是链加，第二种情况拆开后是链加对应的深度和链加固定数。
- 一样的维护方式，因为上面所有的情况有点多，讨论起来可能会有点麻烦，代码比较长。

QOJ1163. Another Tree Queries Problem

- 然后第二个操作 $(2, u, v)$ 。
- 同理也可以拆成 lca 到根路径上的点和 (u, v) 最短路径上的点。
- 同样的前面的部分也是链加，第二种情况拆开后是链加对应的深度和链加固定数。
- 一样的维护方式，因为上面所有的情况有点多，讨论起来可能会有点麻烦，代码比较长。
- 时间复杂度 $\Theta(n \log^2 n)$ 。

LOJ3914. 「PA 2022」 Pótno

- 考虑有一个 2 行 n 列的网格图，我们认为第一列和最后一列相邻。
- 如果两个方格有一条公共边，我们认为它们是相邻的，即它们要么在同一列，要么在同一行的相邻列。
- 所有格子里面填上了 $1 \sim 2n$ 的数，任意两个格子填的数不同。
- 称区间 $[l, r] (1 \leq l \leq r \leq 2n)$ 的权值为编号 $[l, r]$ 的格子所形成的连通块个数。
- 给定 k ，对于每个 $v \in 1, 2, \dots, k$ 求区间权值为 k 的区间个数。
- 数据范围： $n \leq 10^5, k \leq 10$ 。

LOJ3914. 「PA 2022」 Pótno

- 考虑网格图是一个平面图，所以我们使用平面图欧拉定理：
 $|V| + |E| + |R| = 1 + C$ 其中 $|V|$ 是顶点数， $|E|$ 是边数， $|R|$ 是面数， C 是连通块数。

LOJ3914. 「PA 2022」 Pótno

- 考虑网格图是一个平面图，所以我们使用平面图欧拉定理： $|V| + |E| + |R| = 1 + C$ 其中 $|V|$ 是顶点数， $|E|$ 是边数， $|R|$ 是面数， C 是连通块数。
- 具体的运用到这道题，考虑对于任意区间 $[l, r]$ 的权值为 k ，当且仅当 $k = v_{1 \times 1} - v_{1 \times 2} + v_{2 \times 2}$ 其中 $v_{i \times j}$ 表示该区间包含了多少个 $i \times j$ 的矩形，注意的包含是可以旋转的，也就是 $v_{1 \times 2}$ 既包含了 1×2 矩形，又包含了 2×1 矩形。

LOJ3914. 「PA 2022」 Pótno

- 考虑网格图是一个平面图，所以我们使用平面图欧拉定理： $|V| + |E| + |R| = 1 + C$ 其中 $|V|$ 是顶点数， $|E|$ 是边数， $|R|$ 是面数， C 是连通块数。
- 具体的运用到这道题，考虑对于任意区间 $[l, r]$ 的权值为 k ，当且仅当 $k = v_{1 \times 1} - v_{1 \times 2} + v_{2 \times 2}$ 其中 $v_{i \times j}$ 表示该区间包含了多少个 $i \times j$ 的矩形，注意的包含是可以旋转的，也就是 $v_{1 \times 2}$ 既包含了 1×2 矩形，又包含了 2×1 矩形。
- 于是我们用线段树维护扫面先的过程，针对 k 的限制，在线段树的节点上维护前 k 小值以及分别的个数即可。

LOJ3914. 「PA 2022」 Pótno

- 考虑网格图是一个平面图，所以我们使用平面图欧拉定理： $|V| + |E| + |R| = 1 + C$ 其中 $|V|$ 是顶点数， $|E|$ 是边数， $|R|$ 是面数， C 是连通块数。
- 具体的运用到这道题，考虑对于任意区间 $[l, r]$ 的权值为 k ，当且仅当 $k = v_{1 \times 1} - v_{1 \times 2} + v_{2 \times 2}$ 其中 $v_{i \times j}$ 表示该区间包含了多少个 $i \times j$ 的矩形，注意的包含是可以旋转的，也就是 $v_{1 \times 2}$ 既包含了 1×2 矩形，又包含了 2×1 矩形。
- 于是我们用线段树维护扫面先的过程，针对 k 的限制，在线段树的节点上维护前 k 小值以及分别的个数即可。
- 时间复杂度 $\Theta(nk \log n)$ 。

LOJ576. 「LibreOJ NOI Round 2」 签到游戏

- 有一个长度为 n 的序列 a ，但是你不知道 a 的具体值。
- 同时公开一个长度为 n 的序列 b 。
- 一次询问需要花费 $\gcd_{i=1}^r b_i$ 的代价知道 $\sum_{i=1}^r a_i$ 的值。
- 你需要用最小的代价得到所有 a_i 的具体值，输出最小代价，不需要输出方案。
- 有 m 次对 b 的单点修改。
- 数据范围： $n, m \leq 10^5, b_i \leq 10^9$ 。

LOJ576. 「LibreOJ NOI Round 2」 签到游戏

- 考虑假设 s 是 a 的前缀和数组。

LOJ576. 「LibreOJ NOI Round 2」 签到游戏

- 考虑假设 s 是 a 的前缀和数组。
- 那么一次询问可以知道 $s_r - s_{l-1}$ 的值，我们把这个看成 $(l-1, r)$ 连了一条边。

LOJ576. 「LibreOJ NOI Round 2」 签到游戏

- 考虑假设 s 是 a 的前缀和数组。
- 那么一次询问可以知道 $s_r - s_{l-1}$ 的值，我们把这个看成 $(l-1, r)$ 连了一条边。
- 那么一个方案能得到所有的 a 就等价于 $(0 \dots n)$ 联通。

LOJ576. 「LibreOJ NOI Round 2」 签到游戏

- 考虑假设 s 是 a 的前缀和数组。
- 那么一次询问可以知道 $s_r - s_{l-1}$ 的值，我们把这个看成 $(l-1, r)$ 连了一条边。
- 那么一个方案能得到所有的 a 就等价于 $(0 \dots n)$ 联通。
- 所以相当于要求一棵带边权最小生成树。注意到边权是 $\gcd$ ，所以区间越大边权就会越小。

LOJ576. 「LibreOJ NOI Round 2」 签到游戏

- 考虑假设 s 是 a 的前缀和数组。
- 那么一次询问可以知道 $s_r - s_{l-1}$ 的值，我们把这个看成 $(l-1, r)$ 连了一条边。
- 那么一个方案能得到所有的 a 就等价于 $(0 \dots n)$ 联通。
- 所以相当于要求一棵带边权最小生成树。注意到边权是 $\gcd$ ，所以区间越大边权就会越小。
- 于是我们所有点都只会和 0 或者 n 连边。

LOJ576. 「LibreOJ NOI Round 2」 签到游戏

- 考虑假设 s 是 a 的前缀和数组。
- 那么一次询问可以知道 $s_r - s_{l-1}$ 的值，我们把这个看成 $(l-1, r)$ 连了一条边。
- 那么一个方案能得到所有的 a 就等价于 $(0 \dots n)$ 联通。
- 所以相当于要求一棵带边权最小生成树。注意到边权是 $\gcd$ ，所以区间越大边权就会越小。
- 于是我们所有点都只会和 0 或者 n 连边。
- 更进一步的，最后的方案是一段前缀与 0 连边，一段后缀与 n 连边。而分界点就是第一个满足 $\gcd_{i=1}^x b_i \leq \gcd_{i=x}^n b_i$ 的位置。

LOJ576. 「LibreOJ NOI Round 2」 签到游戏

- 所以一个最暴力的办法就是二分找到分界点，然后对于前缀后缀分别计算答案。

LOJ576. 「LibreOJ NOI Round 2」 签到游戏

- 所以一个最暴力的办法就是二分找到分界点，然后对于前缀后缀分别计算答案。
- 这样的时间复杂度是 $\Theta(n \log^3 n)$ 。虽然经过精细的实现也许这个复杂度足以通过这道题，但是该算法还有优化的空间。

LOJ576. 「LibreOJ NOI Round 2」 签到游戏

- 所以一个最暴力的办法就是二分找到分界点，然后对于前缀后缀分别计算答案。
- 这样的时间复杂度是 $\Theta(n \log^3 n)$ 。虽然经过精细的实现也许这个复杂度足以通过这道题，但是该算法还有优化的空间。
- 优化的办法也十分的套路，把线段树 + 二分直接变成线段树二分，时间复杂度自然就少了一个 $\log$ ，变成 $\Theta(n \log^2 n)$

Thanks!